

# Lecture 12

## EE 421 / CS 425

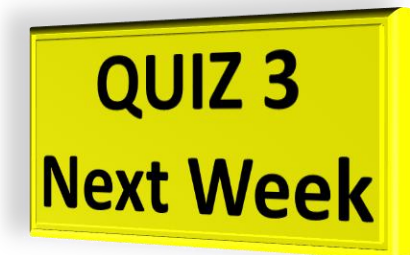
# Digital System Design

Fall 2026

Shahid Masud

# Topics

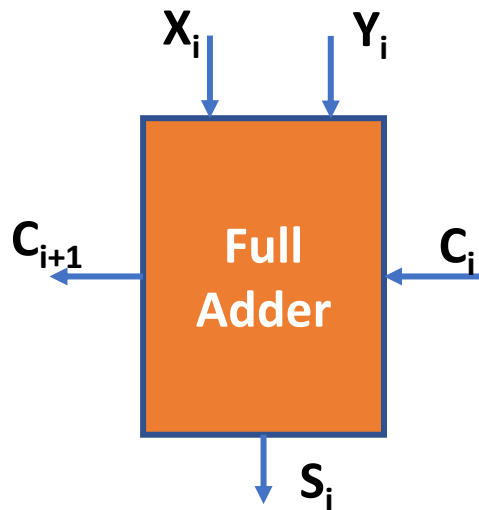
- Quick Recap – Full Adder and Multi-bit Adder Designs
- -----
- Signed Arithmetic
- Overflow Detection
- 2's Complement Arithmetic for Signed Numbers
- Binary Multipliers – timing issues
- Parallel Array Multiplier



# Full Adder Circuit

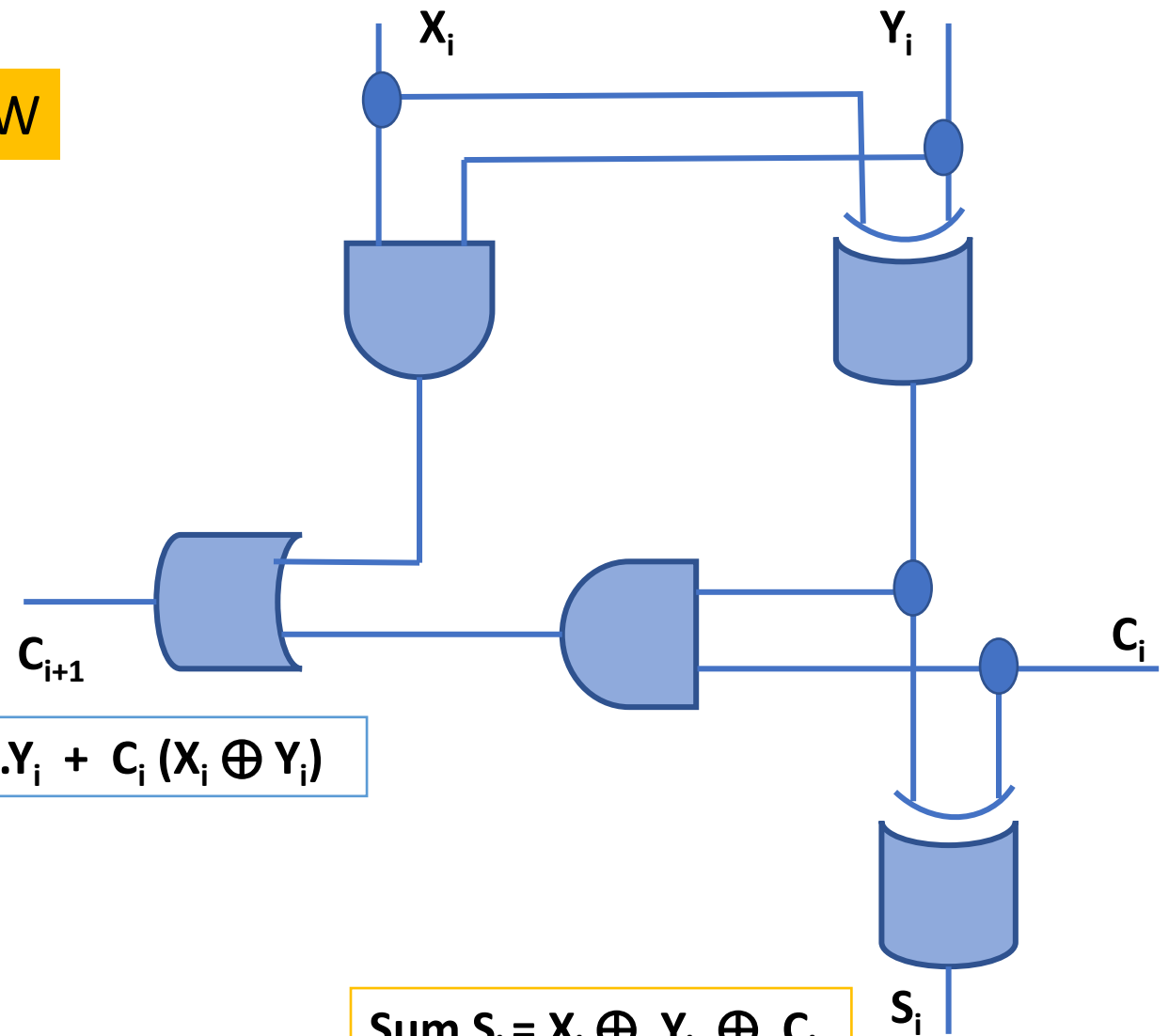
$$\text{Sum } S_i = X_i \oplus Y_i \oplus C_i$$

$$C_{i+1} = X_i \cdot Y_i + C_i (X_i \oplus Y_i)$$



REVIEW

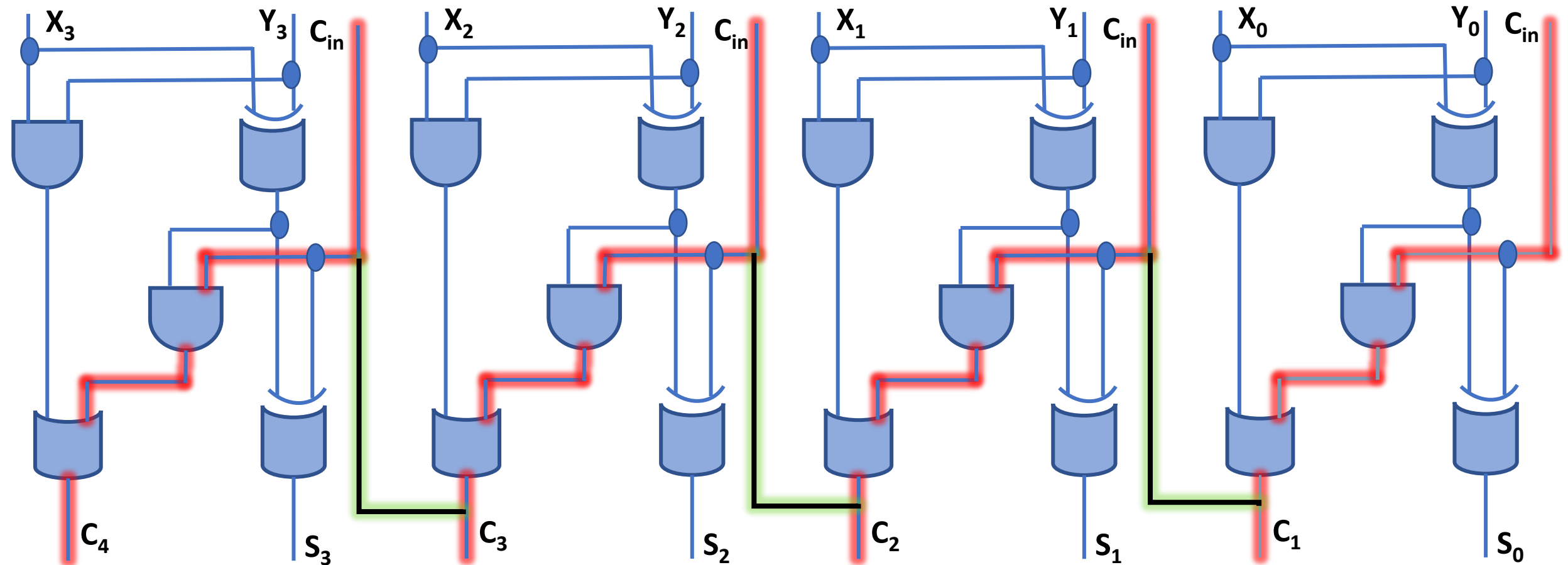
$$C_{i+1} = X_i \cdot Y_i + C_i (X_i \oplus Y_i)$$



$$\text{Sum } S_i = X_i \oplus Y_i \oplus C_i$$

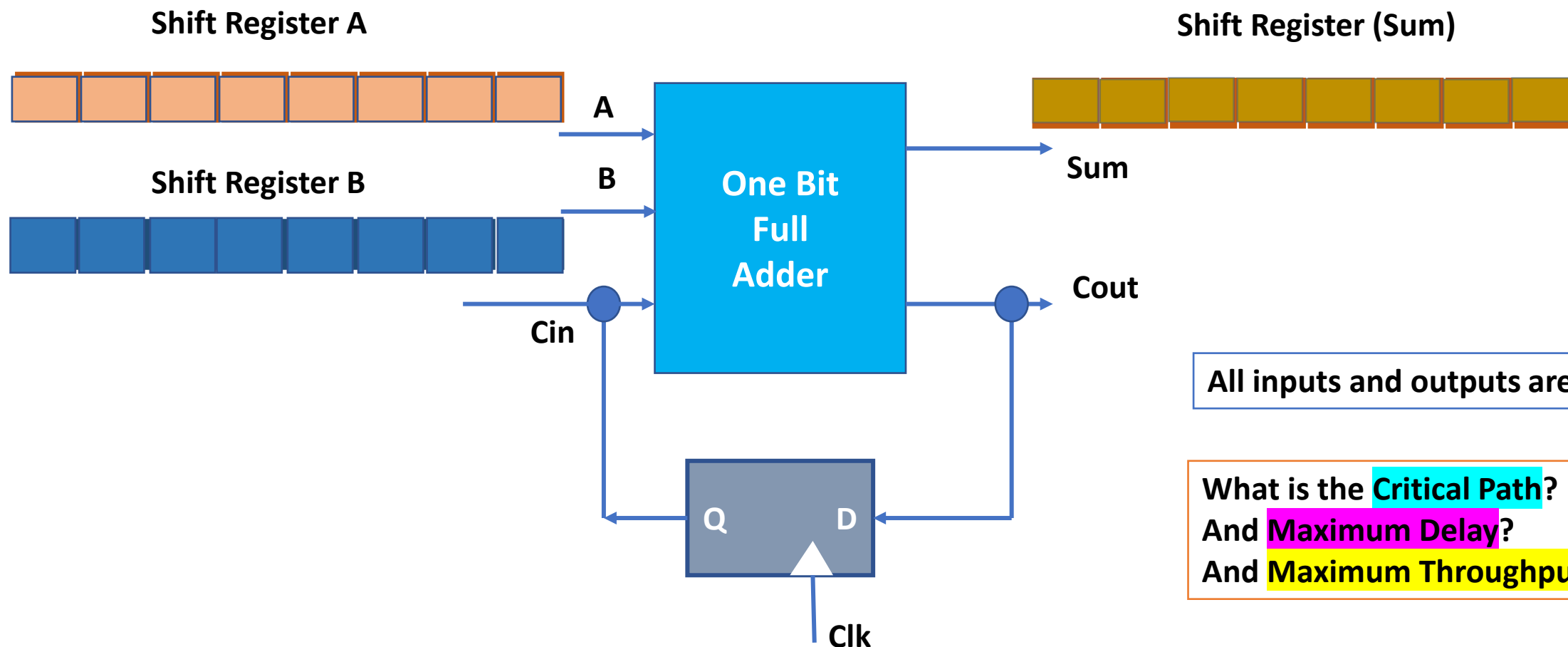
# Compare with Carry Path of Full Adder

REVIEW



# 8-Bit, Bit-Serial Full Adder

REVIEW



All inputs and outputs are one-bit

What is the **Critical Path**?  
And **Maximum Delay**?  
And **Maximum Throughput**?

# Carry Lookahead Adder – A type of fast adder

- In Ripple Carry Adder, the Carry Input  $C_{in}$  propagates through all Adder circuits before reaching the final Carry Out,  $C_{out}$ . Thus, there is Long carry chain that makes the Critical Path worse.
- In Carry Lookahead Adder (CLA), the carry Circuit is separated from the Sum circuit and both work independently. This reduces the number of gates in Critical Path and the Adder can work faster.

Algorithmic Approach

# Delay of 4-Bit CLA Adder

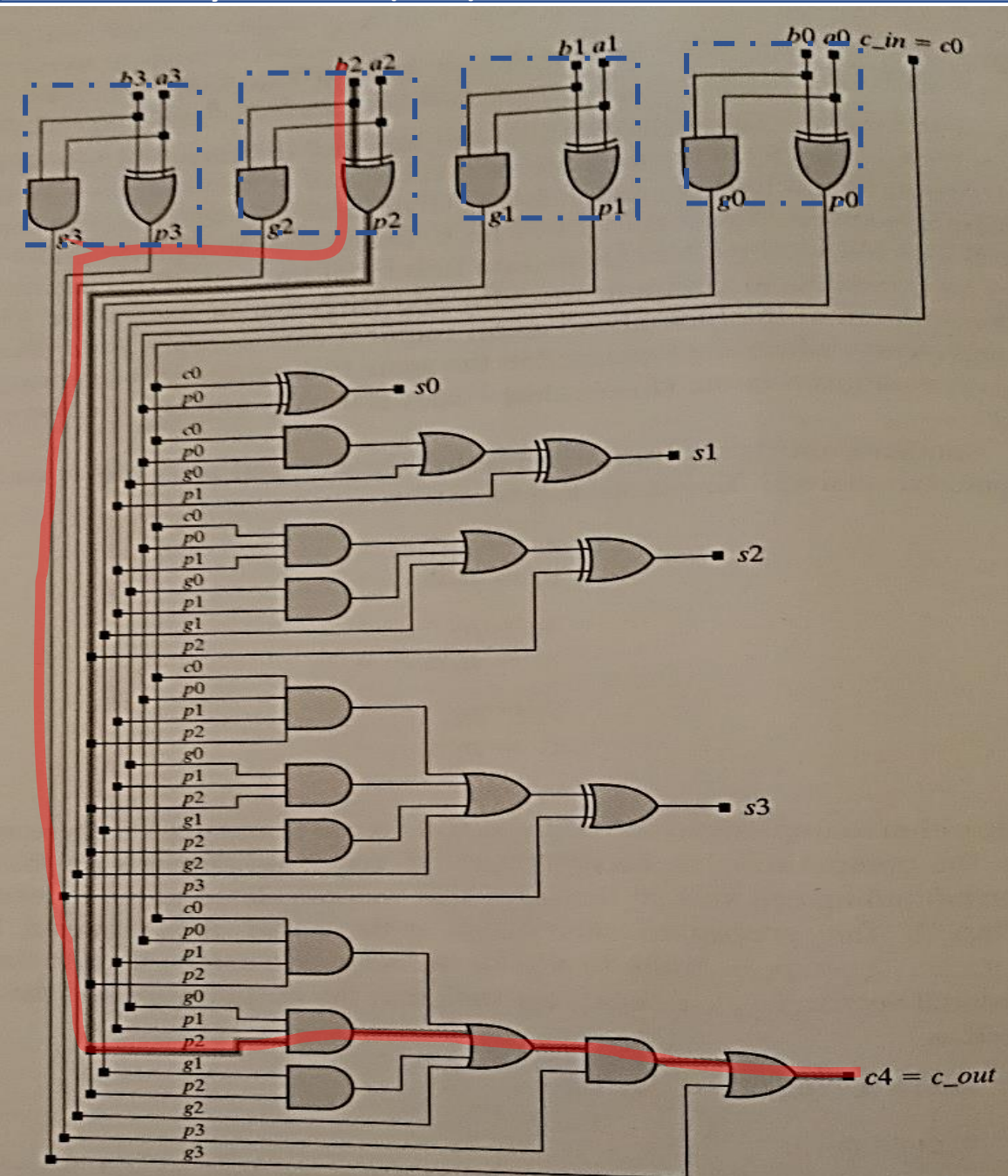
## REVIEW

4-Bit CLA Adder  
Showing Critical Path

Estimate Maximum Clock Speed?

Calculate Sum and Carry Out in the form of P and G

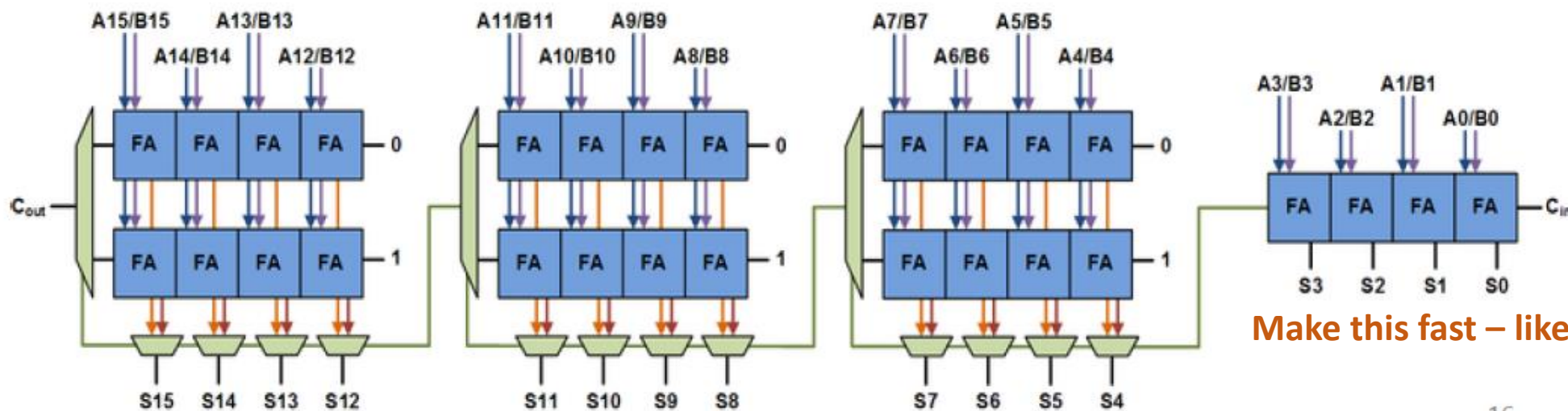
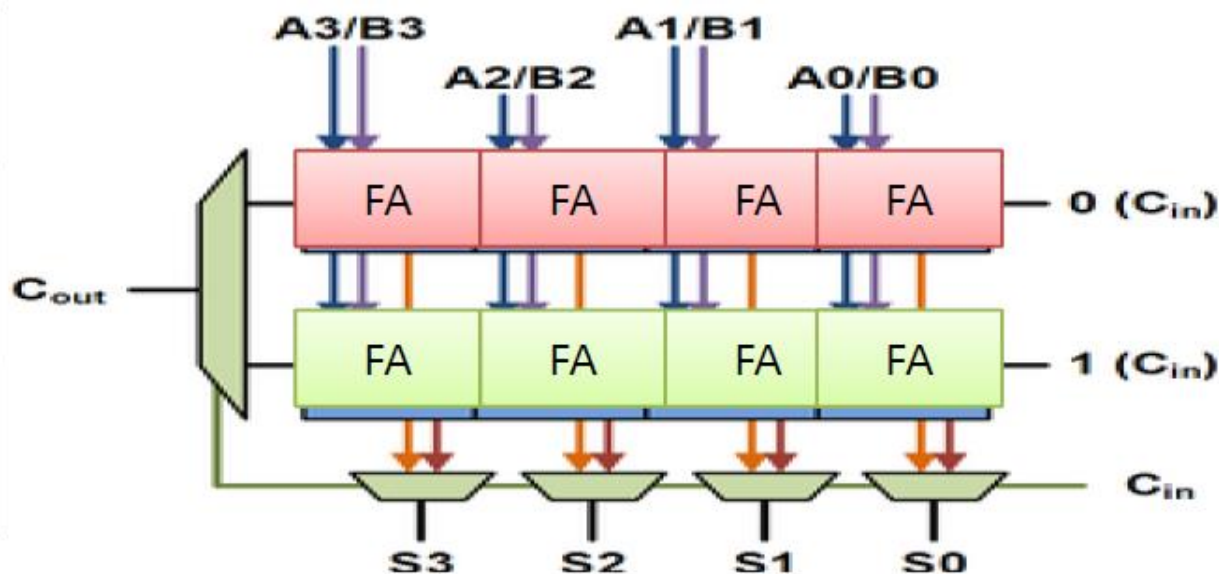
Diagram from Ciletti course textbook



# Carry Select Adder

For a group Sum & Carry is already calculated

Simply select based on carry



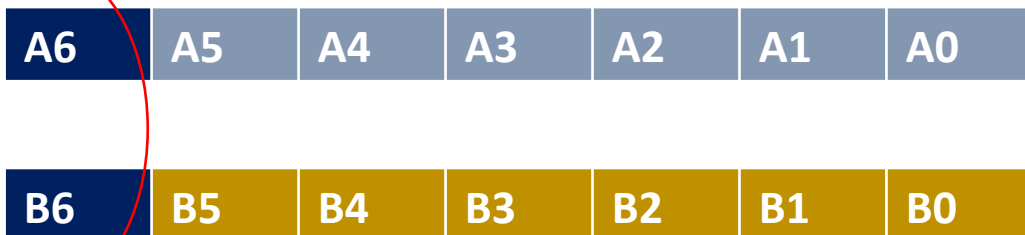
Make this fast – like CLA technique

# Signed Numbers

# Representing Signed Numbers

When we work in a system where both positive and negative numbers can be processed,  
We need to do something about the 'sign' of the numbers

Given two signed numbers A and B



MSB is the 'sign' bit in Signed binary representation of numbers, remaining bits represent the magnitude  
'0' sign bit means positive number and '1' sign bit means negative number

In arithmetic processing, the magnitudes are processed (e.g., +/-/x operations) separately and  
Correct sign is determined and inserted at the end of the processing

# 2's Complement Representation

- Sign-bit becomes a part of the number after taking 2's Complement
- The arithmetic circuit is based on one sign (eg. positive numbers) only
- The sign-bit of the answer guides if the number is positive or negative
- 2'S Complement of the answer is taken at the end to get a positive number

2's Complement of a binary number is processed as follows:

- ❖ Add One or More Extra bits to take care of sign-extension
- ❖ All '0' in extended bits indicate positive number
- ❖ All '1' in extended bits indicate negative number
- ❖ If extended bits are '1'; first take 1's complement, i.e. invert all bits
- ❖ Then add "+1" to the number
- ❖ The answer is 2's Complement of the number

# Example of 2's Complement

E.g. Take 2's Complement of decimal number -45

Binary representation of "+45" is "101101"

Take minimum one extra bit for sign, negative means '1'

So minimum signed binary width of "-45" is "1101101"

|                                              |          |          |          |          |          |          |          |
|----------------------------------------------|----------|----------|----------|----------|----------|----------|----------|
| <b>Signed Binary number</b>                  | <b>1</b> | <b>1</b> | <b>0</b> | <b>1</b> | <b>1</b> | <b>0</b> | <b>1</b> |
| <b>Take 1's Complement (invert all bits)</b> | <b>0</b> | <b>0</b> | <b>1</b> | <b>0</b> | <b>0</b> | <b>1</b> | <b>0</b> |
| <b>Add "1" at LSB</b>                        |          |          |          |          |          | <b>+</b> | <b>1</b> |
|                                              | <b>0</b> | <b>0</b> | <b>1</b> | <b>0</b> | <b>0</b> | <b>1</b> | <b>1</b> |

**2's Complement of "-45" is a 7-bit binary number "0010011"**

# 2's Complement of +45

E.g. Take 2's Complement of decimal number +45

Binary representation of 45 is "101101"

Take minimum extra bit for sign, positive means '0'

So minimum signed binary width of "+45" is "0101101"

|                                              |          |          |          |          |          |          |          |
|----------------------------------------------|----------|----------|----------|----------|----------|----------|----------|
| <b>Signed Binary number</b>                  | <b>0</b> | <b>1</b> | <b>0</b> | <b>1</b> | <b>1</b> | <b>0</b> | <b>1</b> |
| <b>Take 1's Complement (invert all bits)</b> | <b>1</b> | <b>0</b> | <b>1</b> | <b>0</b> | <b>0</b> | <b>1</b> | <b>0</b> |
| <b>Add "1" at LSB</b>                        |          |          |          |          |          | <b>+</b> | <b>1</b> |
| <b>Result</b>                                | <b>1</b> | <b>0</b> | <b>1</b> | <b>0</b> | <b>0</b> | <b>1</b> | <b>1</b> |

2's Complement of "+45" is a 7-bit binary number "1010011"

The '1' at MSB indicates it is now a negative number, after taking 2's Complement of a positive number

# Another quick method for 2's Complement

|                                                                  |                           |   |   |   |                                 |                |                     |
|------------------------------------------------------------------|---------------------------|---|---|---|---------------------------------|----------------|---------------------|
| Decimal number = +46                                             | <div>Extra sign bit</div> |   |   |   |                                 |                |                     |
| <b>Given Binary number</b>                                       | 0                         | 1 | 0 | 1 | 1                               | 1              | 0                   |
| <b>Examine from Right to Left (LSB to MSB)</b>                   | 1                         | 0 | 1 | 0 | 0                               | 1              | 0                   |
| <b>When you encounter first '1', invert all bits on the left</b> | 0                         | 1 | 0 | 1 | 1                               | 1              | 0                   |
|                                                                  |                           |   |   |   | Invert this bit and all to left | First '1' seen | LSB is '0', go left |
| <b>2's Complement Answer</b>                                     | 1                         | 0 | 1 | 0 | 0                               | 1              | 0                   |

Extra bits for sign have to be specified before conversion

All extra bits occupy same value, '0' for positive and '1' for negative, bit replication

# 2's Complement Addition Examples

Case 1: Both numbers positive  
4-Bit numbers with one bit for sign

Add "+9" and "+4"

|    |   |   |   |   |   |
|----|---|---|---|---|---|
| +9 | 0 | 1 | 0 | 0 | 1 |
| +4 | 0 | 0 | 1 | 0 | 0 |
|    | 0 | 1 | 1 | 0 | 1 |

Sign bit

Answer = "1101" = "+13"

Case 2: Positive numbers and smaller negative number  
4-Bit numbers with one bit for sign

Add "+9", "-4"; using 5 bits total

2's Complement of 4 (00100) is (11100)

|        |          |   |   |   |   |
|--------|----------|---|---|---|---|
| +9     | 0, 1 Cin | 1 | 0 | 0 | 1 |
| -4     | 1        | 1 | 1 | 0 | 0 |
| 1 Cout | 0        | 0 | 1 | 0 | 1 |

Extra bit, Sign bit

Carry out

Answer = "00101" = "+5"

Cout is beyond 5 bits, hence discarded

# 2's Complement more addition cases

Case 3: Positive number and a larger negative number  
4-Bit numbers with one bit for sign

Add "+4", "-9"; using 5 bits total

2's Complement of 9 (01001) is (10111)

|    |   |          |   |   |   |
|----|---|----------|---|---|---|
| +4 | 0 | 0, Cin 1 | 1 | 0 | 0 |
| -9 | 1 | 0        | 1 | 1 | 1 |
|    | 1 | 1        | 0 | 1 | 1 |

Sign bit

Answer = "11011"

Take 2's Complement = "00101" = "-5"

Case 4: Both negative numbers  
4-Bit numbers with one bit for sign

Add "-9", "-4"; using 5 bits total

2's Complement of 4 (00100) is (11100)

2's Complement of 9 (01001) is (10111)

|        |          |          |   |   |   |
|--------|----------|----------|---|---|---|
| -9     | 1, Cin 1 | 0, Cin 1 | 1 | 1 | 1 |
| -4     | 1        | 1        | 1 | 0 | 0 |
| 1 Cout | 1        | 0        | 0 | 1 | 1 |

Extra bit, Sign bit

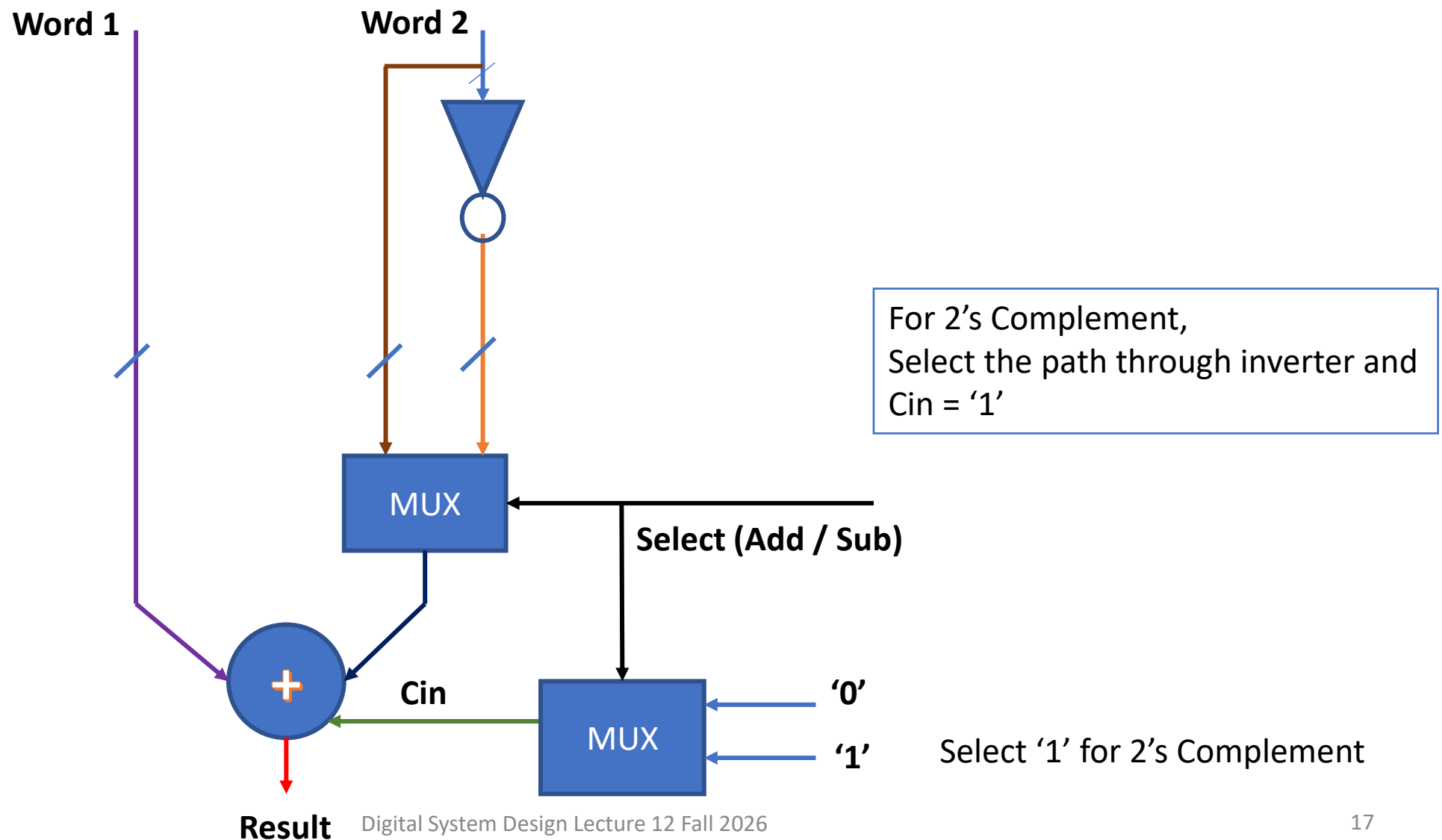
Carry out

Answer = "10011"

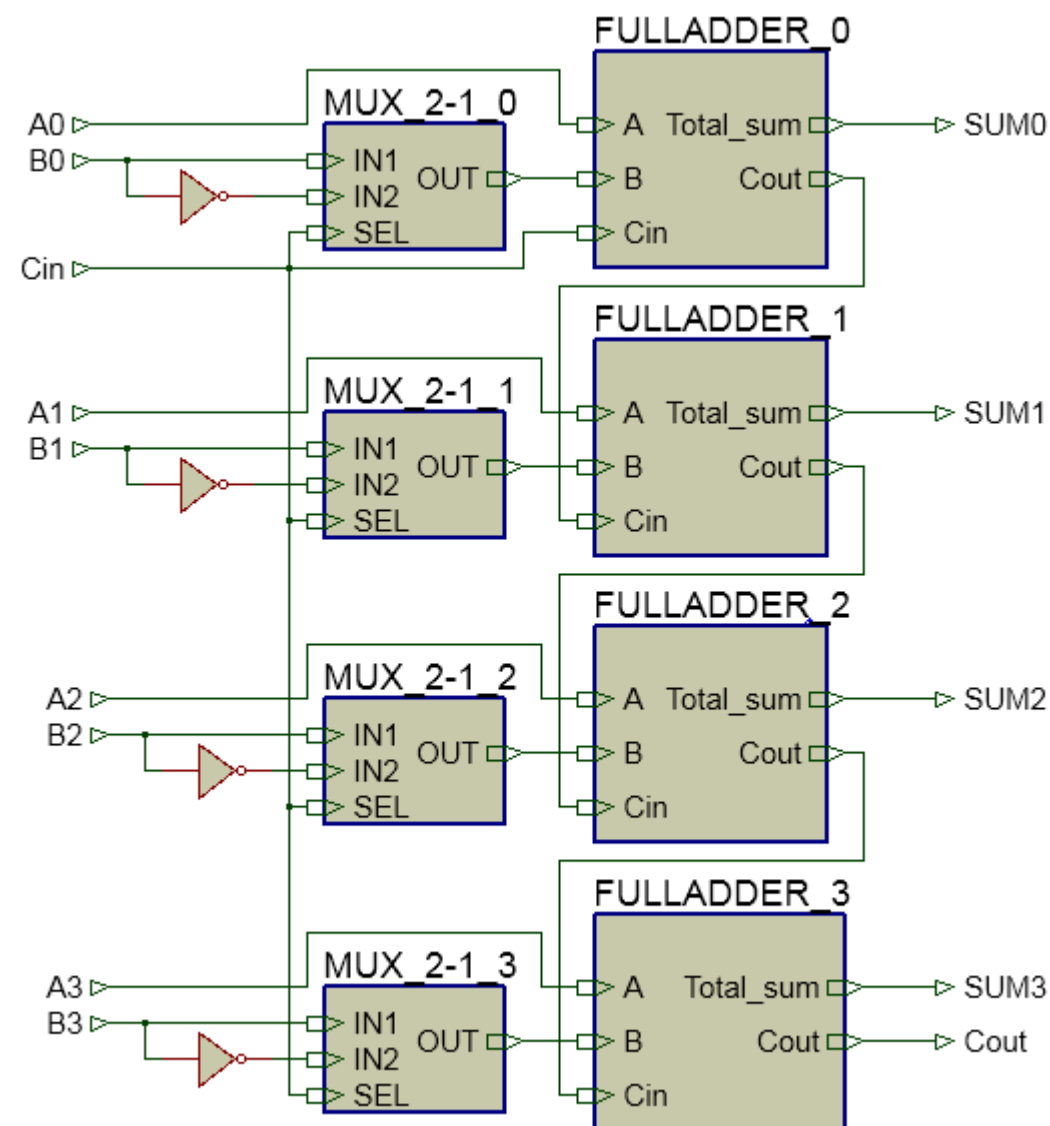
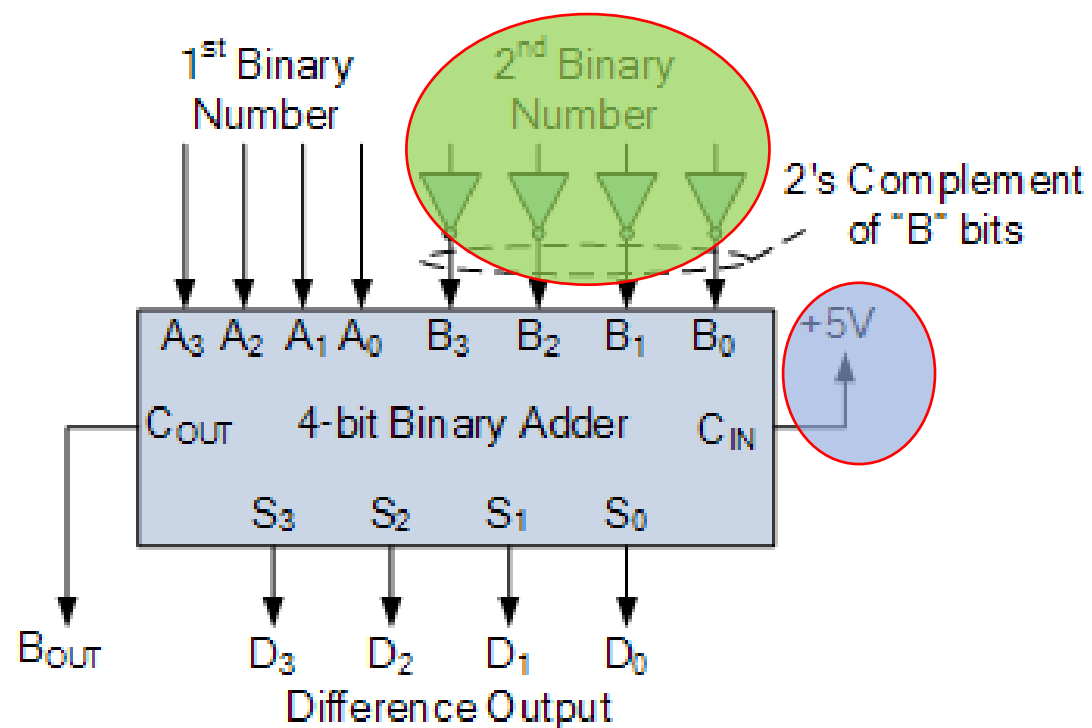
Take 2'Complement = "01101" = "-13"

Cout is beyond 5 bits, hence discarded

# Combined Adder and 2's Complement Subtraction



# 4-Bit Adder/Subtractor Circuit



<https://www.electronics-tutorials.ws/combinational/binary-subtractor.html>

Ref: <https://www.electricaltechnology.org/2018/05/binary-adder-subtractor.html>

# Overflow Condition

Add "+9" and "+8", using 5-bit binary arithmetic circuit

|    |          |   |   |   |   |
|----|----------|---|---|---|---|
| +9 | 0, Cin 1 | 1 | 0 | 0 | 1 |
| +8 | 0        | 1 | 0 | 0 | 0 |
|    | 1        | 0 | 0 | 0 | 1 |

Sign bit

Answer = "10001" = "-1"

Both numbers are positive with '0' sign bit

The answer should be positive too with '0' sign bit

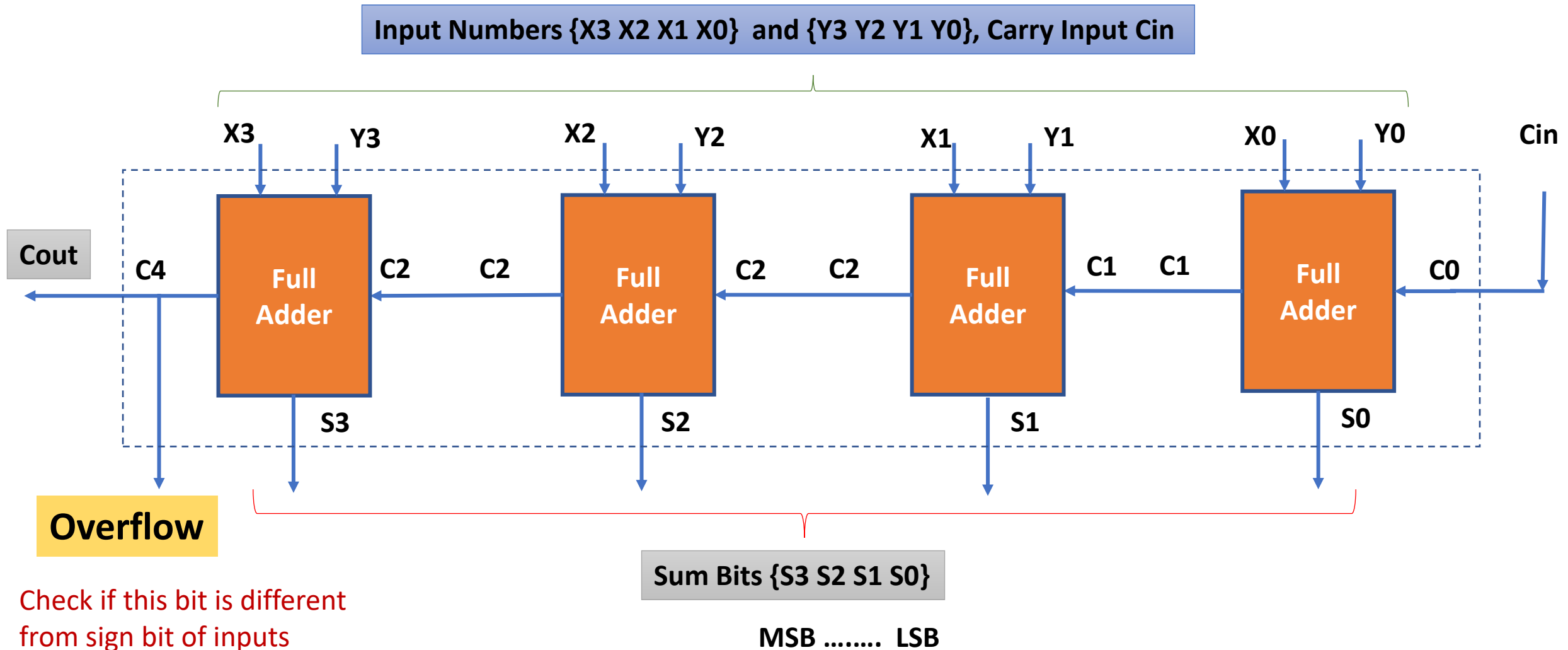
The above is the condition of 'Overflow'. Shows that bits allocated for this arithmetic operation are in-sufficient

The electronic circuits of a processor can easily detect overflow of unsigned binary addition by checking if the carry-out of the leftmost column is a zero or a one. A program might branch to an error handling routine when overflow is detected.

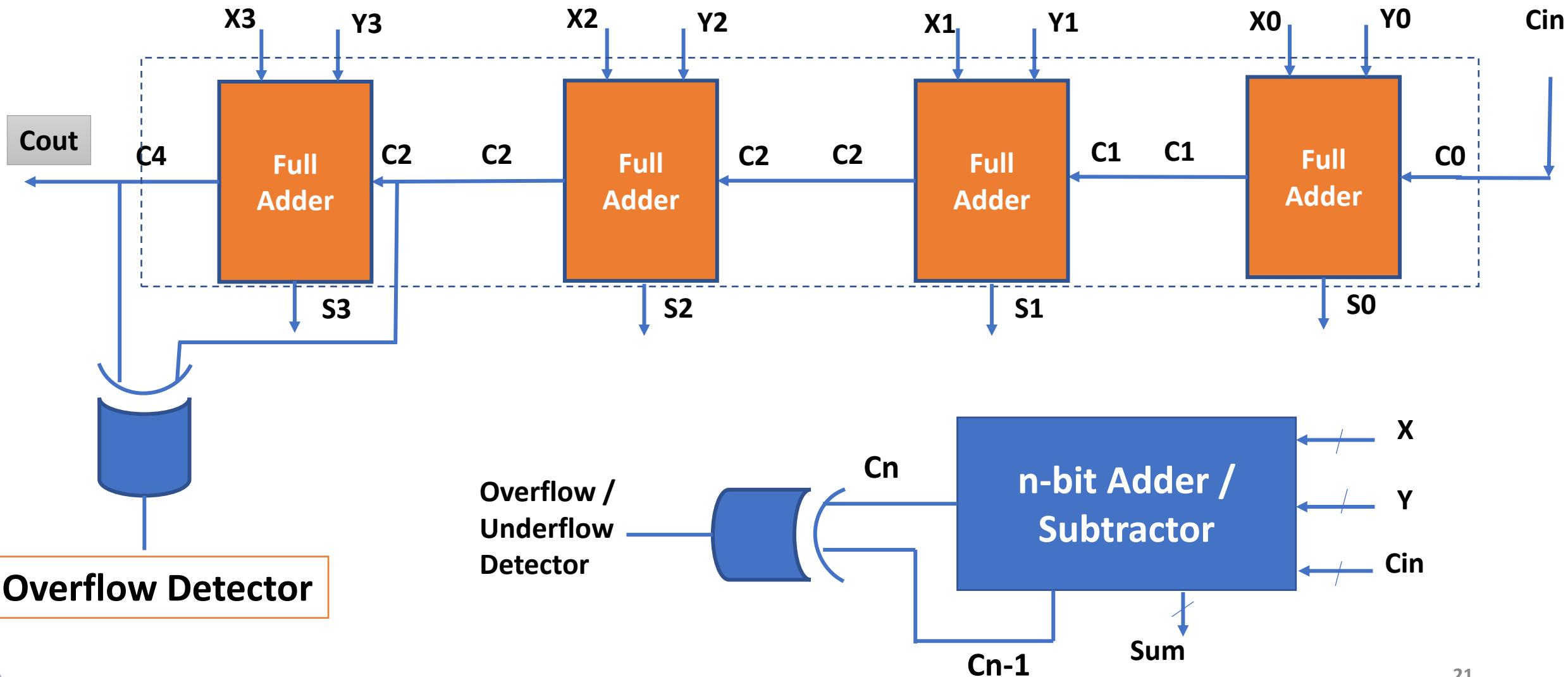
## Overflow in 2's Complement

- If x and y have opposite signs (one is negative, the other is non-negative), then the sum will never overflow. The result will either be x or y or somewhere in between.
- Thus, overflow can only occur when x and y have the same sign.
- One way to detect overflow is to check the sign bit of the sum. If the sign bit of the sum does not match the sign bit of x and y, then there's overflow.
- Suppose x and y both have sign bits with value 1. That means, both representations represent negative numbers. If the sum has sign bit 0, then the result of adding two negative numbers has resulted in a non-negative result, which is clearly wrong. Overflow has occurred.
- Suppose x and y both have sign bits with value 0. That means, both representations represent non-negative numbers. If the sum has sign bit 1, then the result of adding two non-negative numbers has resulted in a negative result, which is clearly wrong. Overflow has occurred.

# Overflow in Un-signed in Binary Adder / Sub



# Overflow in Signed Binary Add / Sub



# Binary Multiplication

# Decimal Multiplication using Pencil and paper



$$\begin{array}{r}
 \phantom{00}943 \\
 \times \phantom{00}18 \\
 \hline
 7544 \\
 + 9430 \\
 \hline
 16974
 \end{array}$$

Keep shifting right

Keep shifting left

# Array Multipliers – Parallel and Serial forms

$$X = \sum_{i=0}^{m-1} X_i \cdot 2^i$$

$$Y = \sum_{j=0}^{n-1} Y_j \cdot 2^j$$

$$P = X \cdot Y = \sum_{i=0}^{m-1} X_i \cdot 2^i \cdot \sum_{j=0}^{n-1} Y_j \cdot 2^j$$

$$P = \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} (X_i Y_j) 2^{i+j}$$

$$P = \sum_{k=0}^{m+n-1} P_k 2^k$$

|      |           |           |           |           |           |           |       |
|------|-----------|-----------|-----------|-----------|-----------|-----------|-------|
|      |           |           | $X_3$     | $X_2$     | $X_1$     | $X_0$     |       |
|      |           |           | $Y_3$     | $Y_2$     | $Y_1$     | $Y_0$     |       |
|      |           |           | $X_3 Y_0$ | $X_2 Y_0$ | $X_1 Y_0$ | $X_0 Y_0$ |       |
|      |           | $X_3 Y_1$ | $X_2 Y_1$ | $X_1 Y_1$ | $X_0 Y_1$ | Nil       |       |
|      | $X_3 Y_2$ | $X_2 Y_2$ | $X_1 Y_2$ | $X_0 Y_2$ | Nil       | Nil       |       |
|      | $X_3 Y_3$ | $X_2 Y_3$ | $X_1 Y_3$ | $X_0 Y_3$ | Nil       | Nil       | Nil   |
| Cout | $P_6$     | $P_5$     | $P_4$     | $P_3$     | $P_2$     | $P_1$     | $P_0$ |

# Complexity of Binary Array Multiplier

|      |          |          |          | $X_3$    | $X_2$    | $X_1$    | $X_0$    |
|------|----------|----------|----------|----------|----------|----------|----------|
|      |          |          |          | $Y_3$    | $Y_2$    | $Y_1$    | $Y_0$    |
|      |          |          |          | $X_3Y_0$ | $X_2Y_0$ | $X_1Y_0$ | $X_0Y_0$ |
|      |          |          | $X_3Y_1$ | $X_2Y_1$ | $X_1Y_1$ | $X_0Y_1$ | 0        |
|      |          | $X_3Y_2$ | $X_2Y_2$ | $X_1Y_2$ | $X_0Y_2$ | 0        | 0        |
|      | $X_3Y_3$ | $X_2Y_3$ | $X_1Y_3$ | $X_0Y_3$ | 0        | 0        | 0        |
| Cout | $P_6$    | $P_5$    | $P_4$    | $P_3$    | $P_2$    | $P_1$    | $P_0$    |

How many AND gates?

How many Adders?

Propagation Delay – Longest Carry Ripple Path

## Complexity and Timing

For an  $n$ -bit x  $n$ -bit multiplier;  
We need:

$n(n-2)$  full adders

$n$  half adders

$n^2$  AND Gates

Worst Case Delay is  $(2n+1) \tau$   
Where  $\tau$  is the worst adder delay

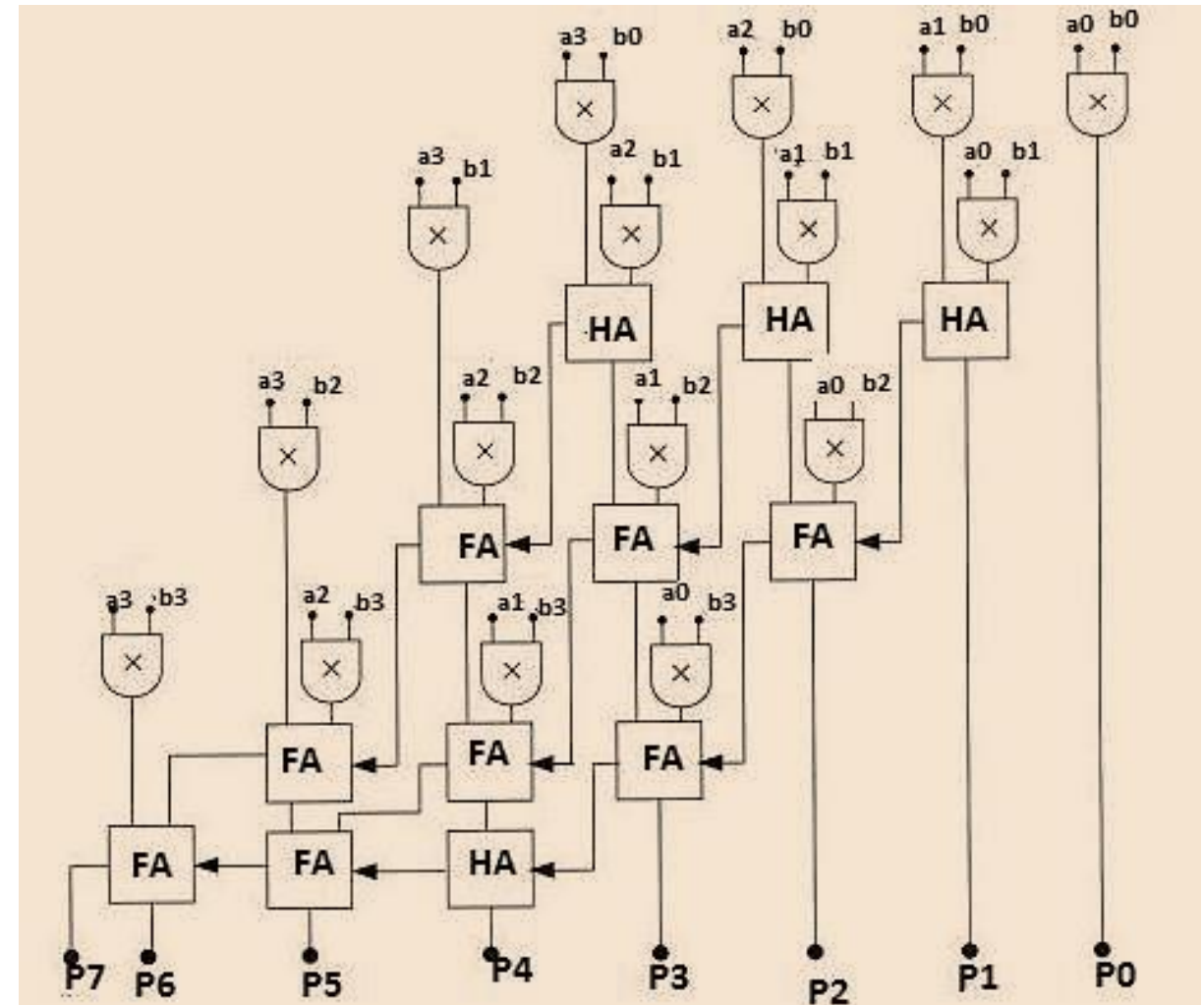
# Question: Delay estimation in Array Multiplier

- Assume, delay through AND Gate is 10ns
- Delay through half-adder is 30ns
- Delay through full-adder is 50ns
- Calculate Critical path delay of a 4-bit array multiplier
- Calculate maximum clock speed for this 4-bit array multiplier



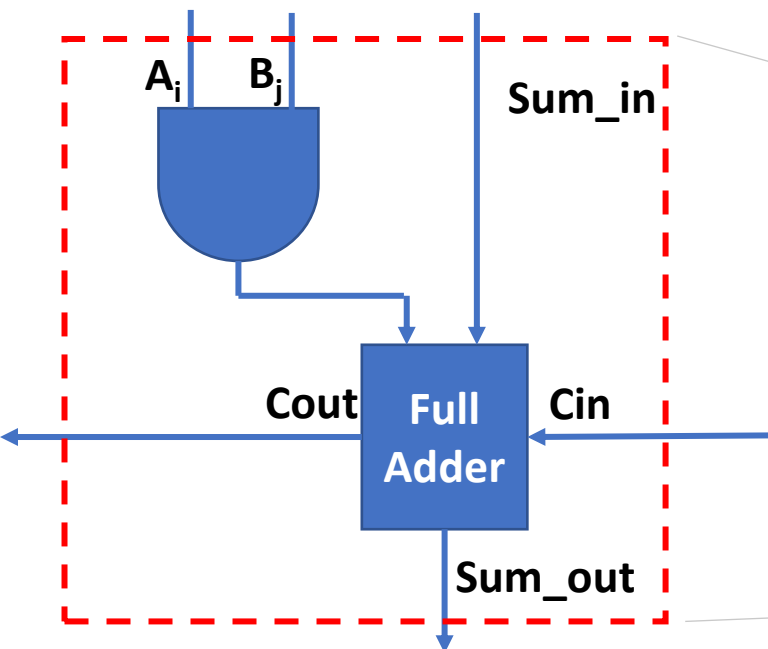
# 4-Bit Array Multiplier connected as AND and ADD

|      |          |          |          | $A_3$    | $A_2$    | $A_1$    | $A_0$    |
|------|----------|----------|----------|----------|----------|----------|----------|
|      |          |          |          | $B_3$    | $B_2$    | $B_1$    | $B_0$    |
|      |          |          |          | $A_3B_0$ | $A_2B_0$ | $A_1B_0$ | $A_0B_0$ |
|      |          | $A_3B_1$ | $A_2B_1$ | $A_1B_1$ | $A_0B_1$ | 0        | 0        |
|      |          | $A_3B_2$ | $A_2B_2$ | $A_1B_2$ | $A_0B_2$ | 0        | 0        |
|      | $A_3B_3$ | $A_2B_3$ | $A_1B_3$ | $A_0B_3$ | 0        | 0        | 0        |
| Cout | $P_6$    | $P_5$    | $P_4$    | $P_3$    | $P_2$    | $P_1$    | $P_0$    |

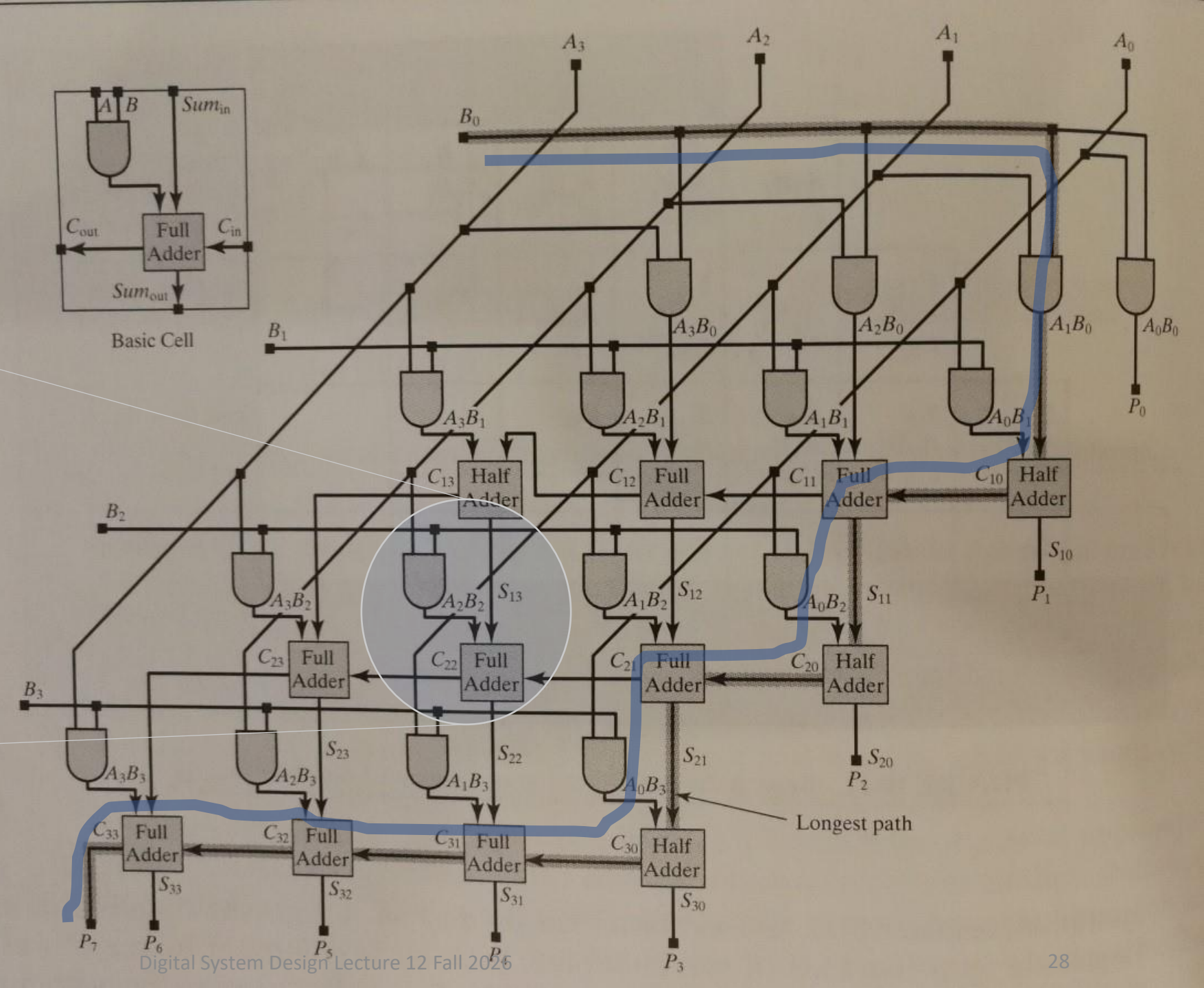


# Array Multiplier Circuit Delays

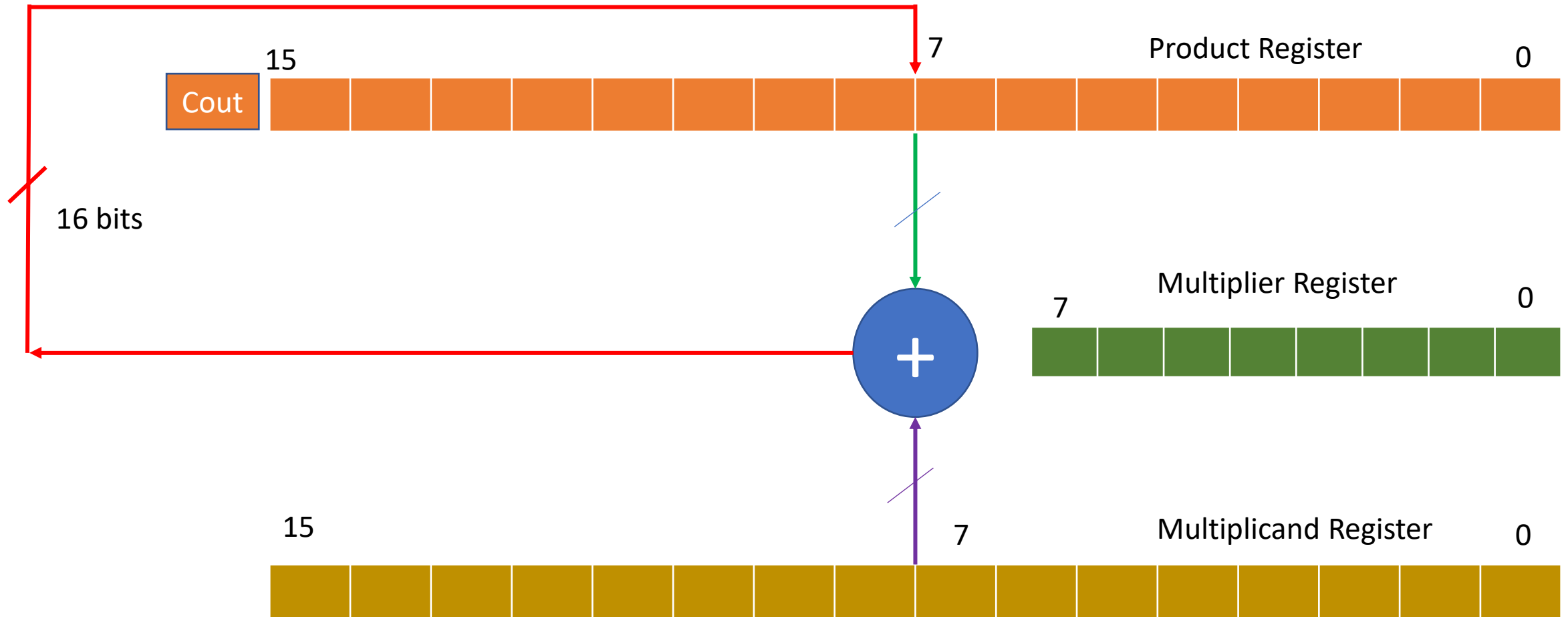
Building Block



Worst case delay path is shaded  
This is **Critical Path**



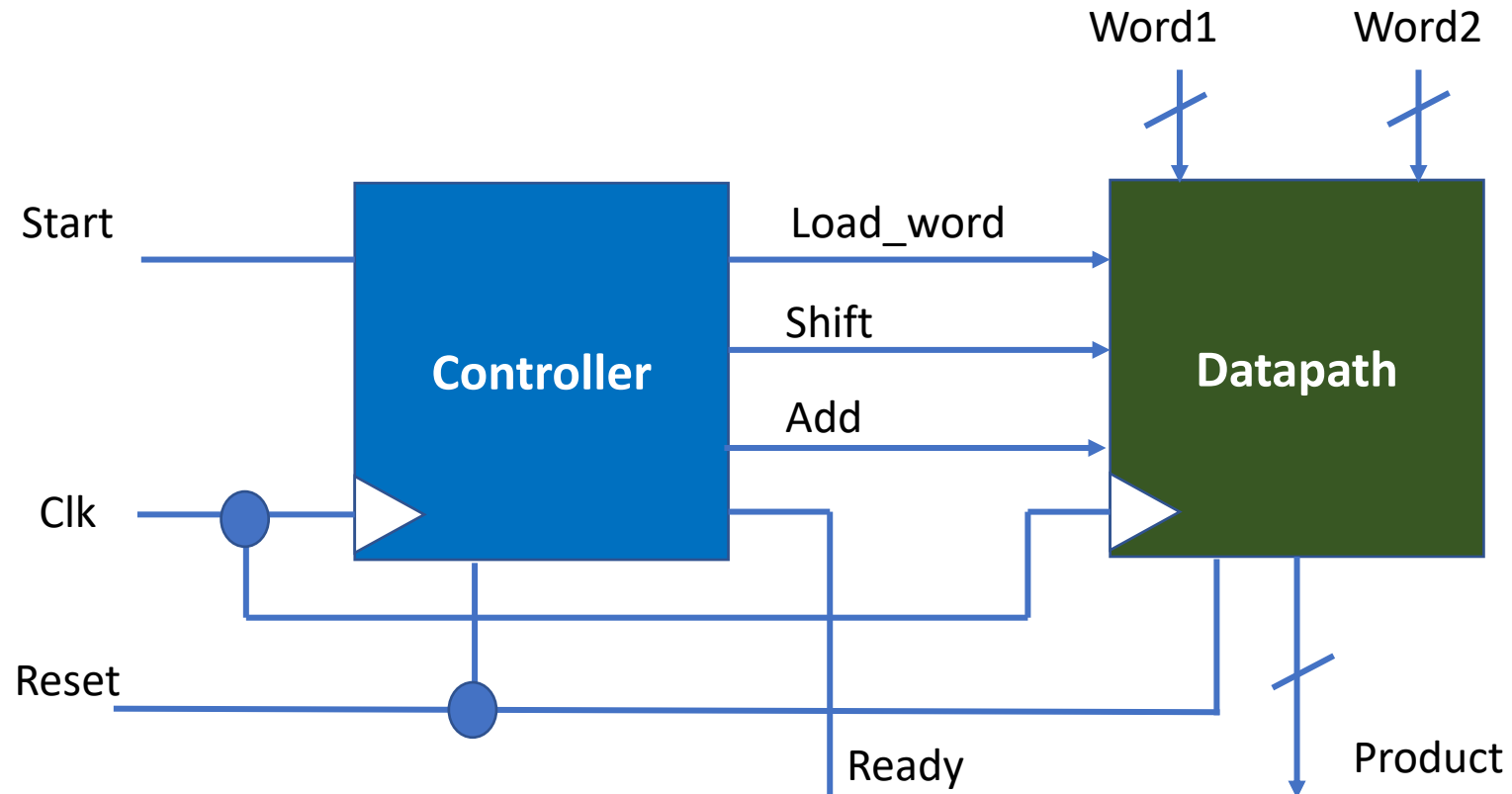
# Operation of Sequential Multiplier



Datapath of a Sequential Multiplier

Digital System Design Lecture 12 Fall 2026

# Data Path Architecture of Sequential Mult



# Register transfer in 8-bit Sequential Multiplier

$$23_{10} \times 215_{10} = 4945_{10}$$

Multiplier register - start

0 0 0 1 0 1 1 1

Shift right

0 0 0 0 1 0 1 1

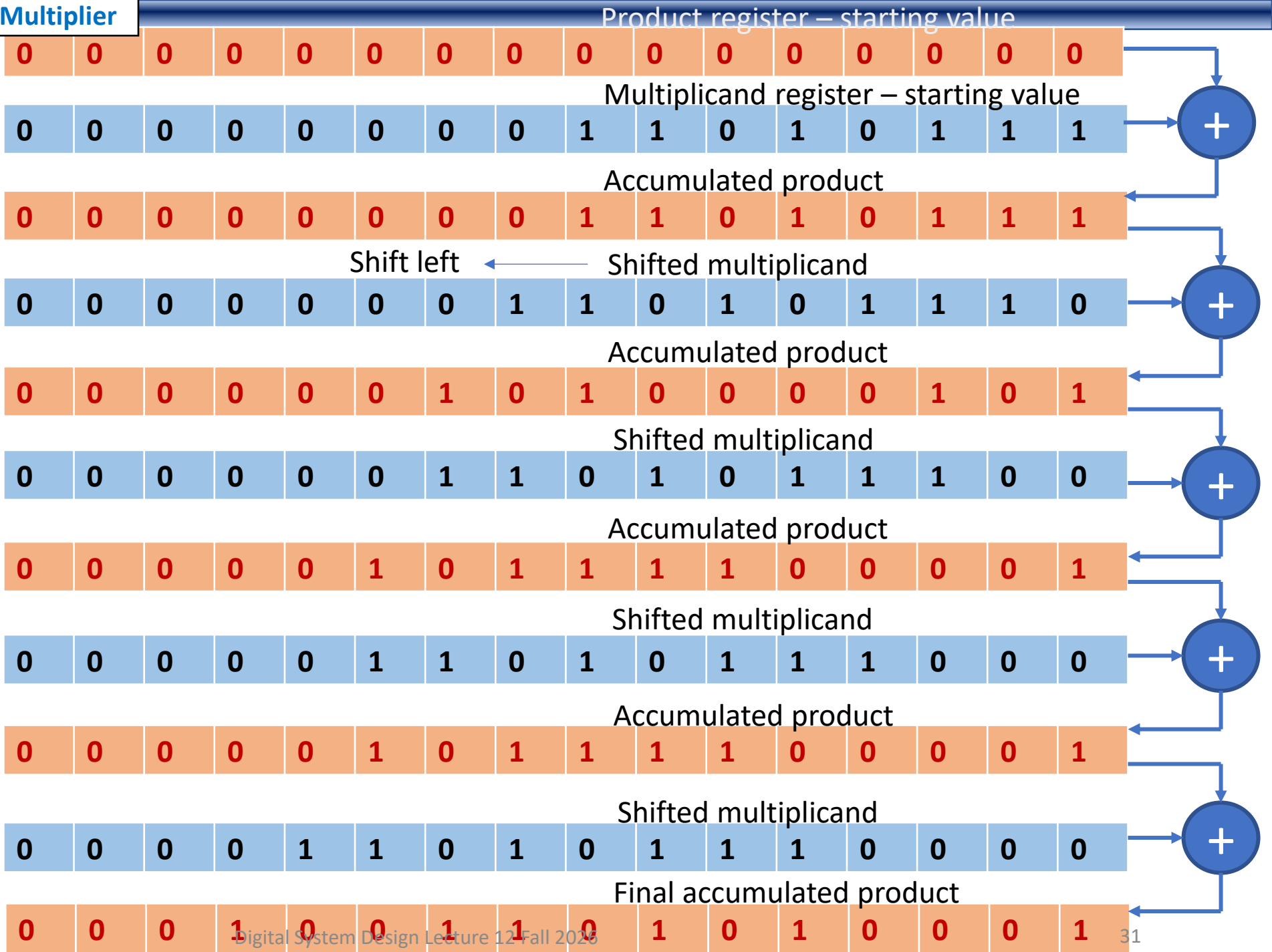
Shift right each cycle

0 0 0 0 0 1 0 1

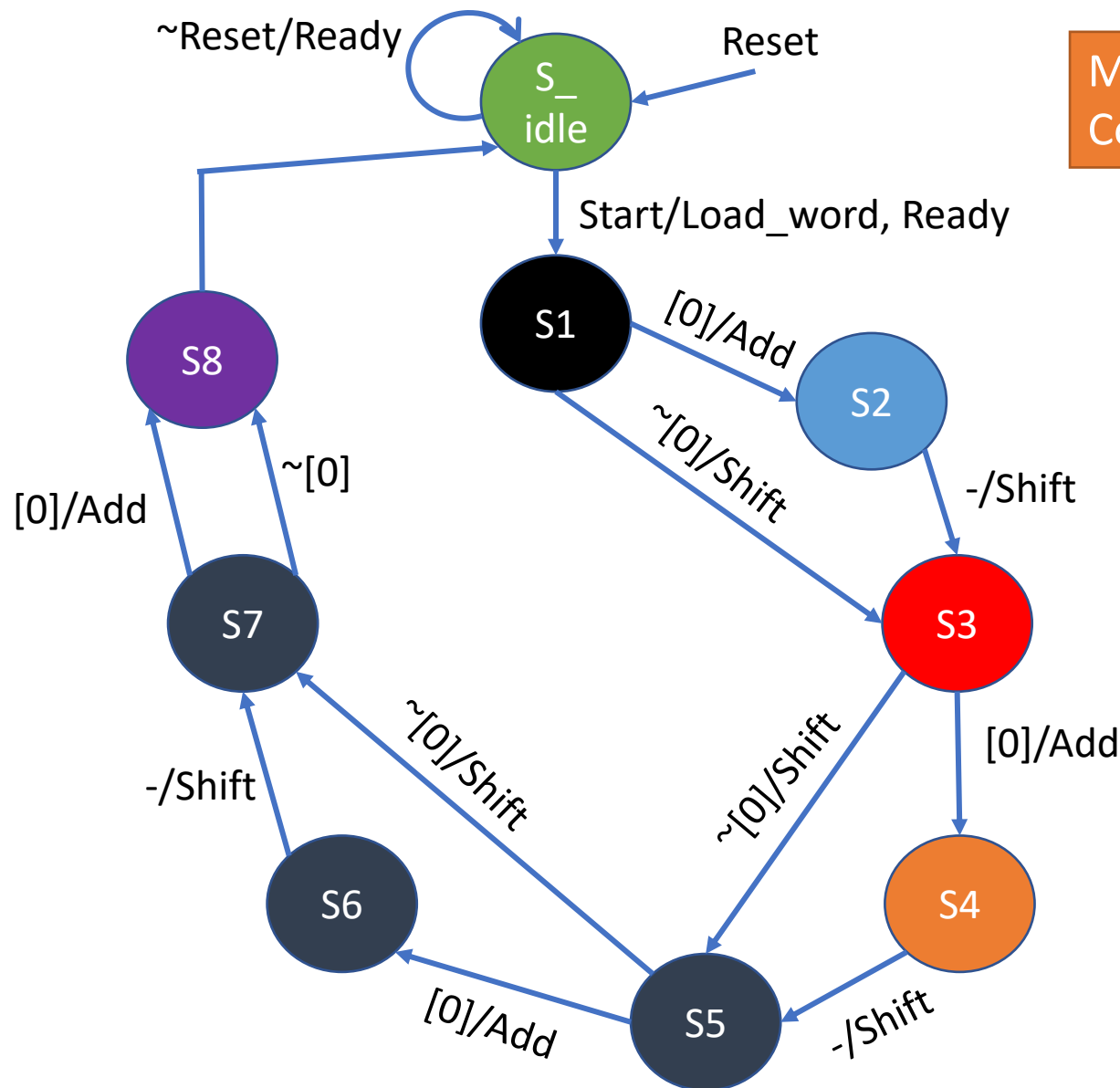
0 0 0 0 0 0 1 0

0 0 0 0 0 0 0 1

0 0 0 0 0 0 0 0

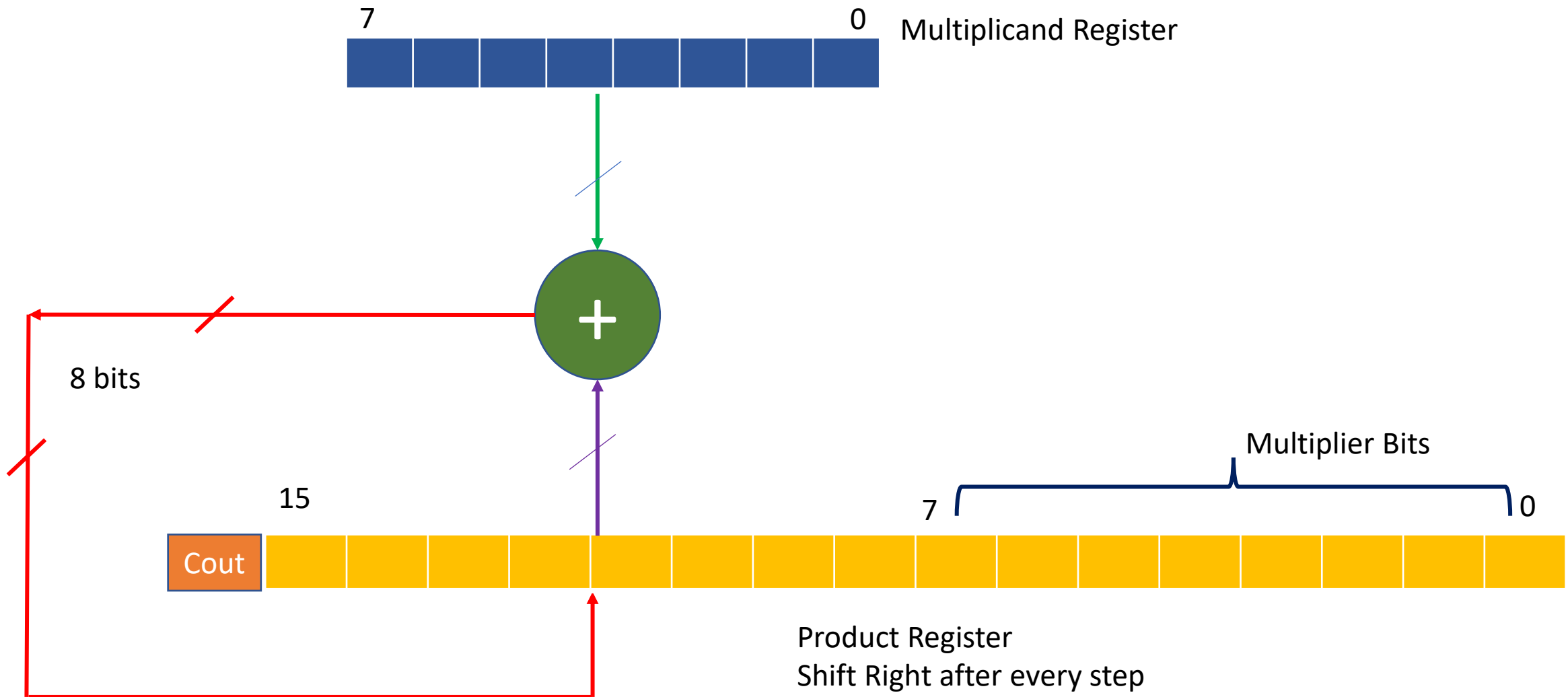


# STG for a 4 Bit Sequential Binary Multiplier

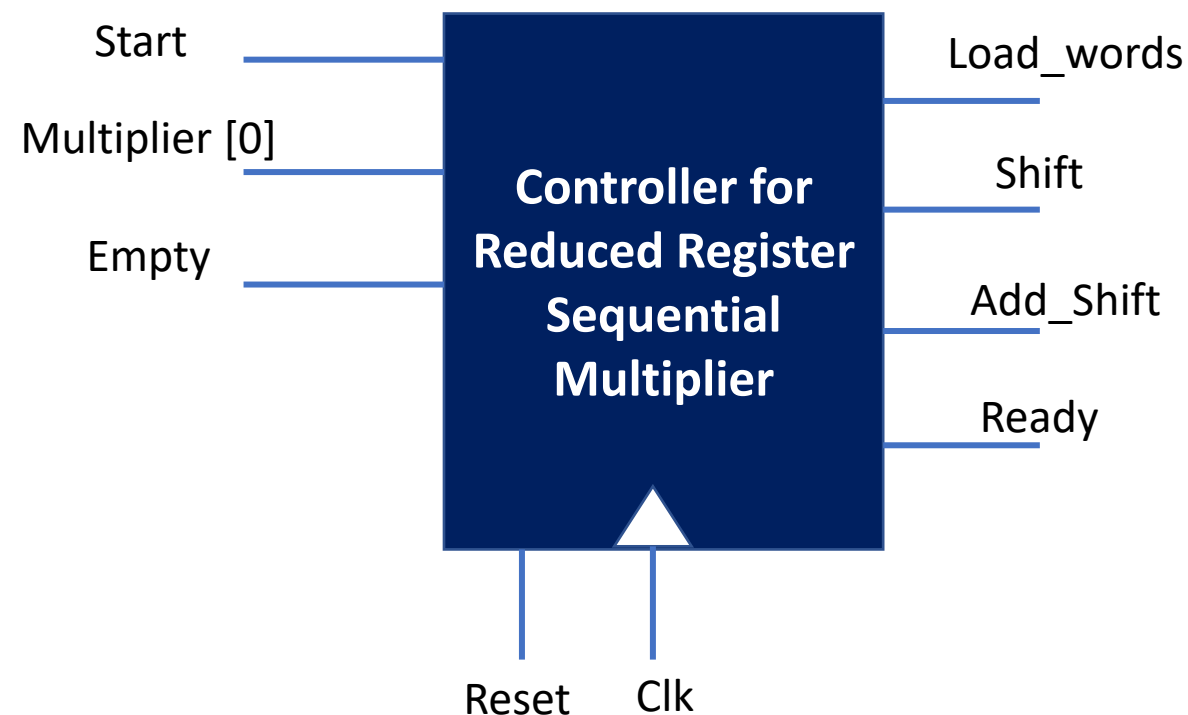
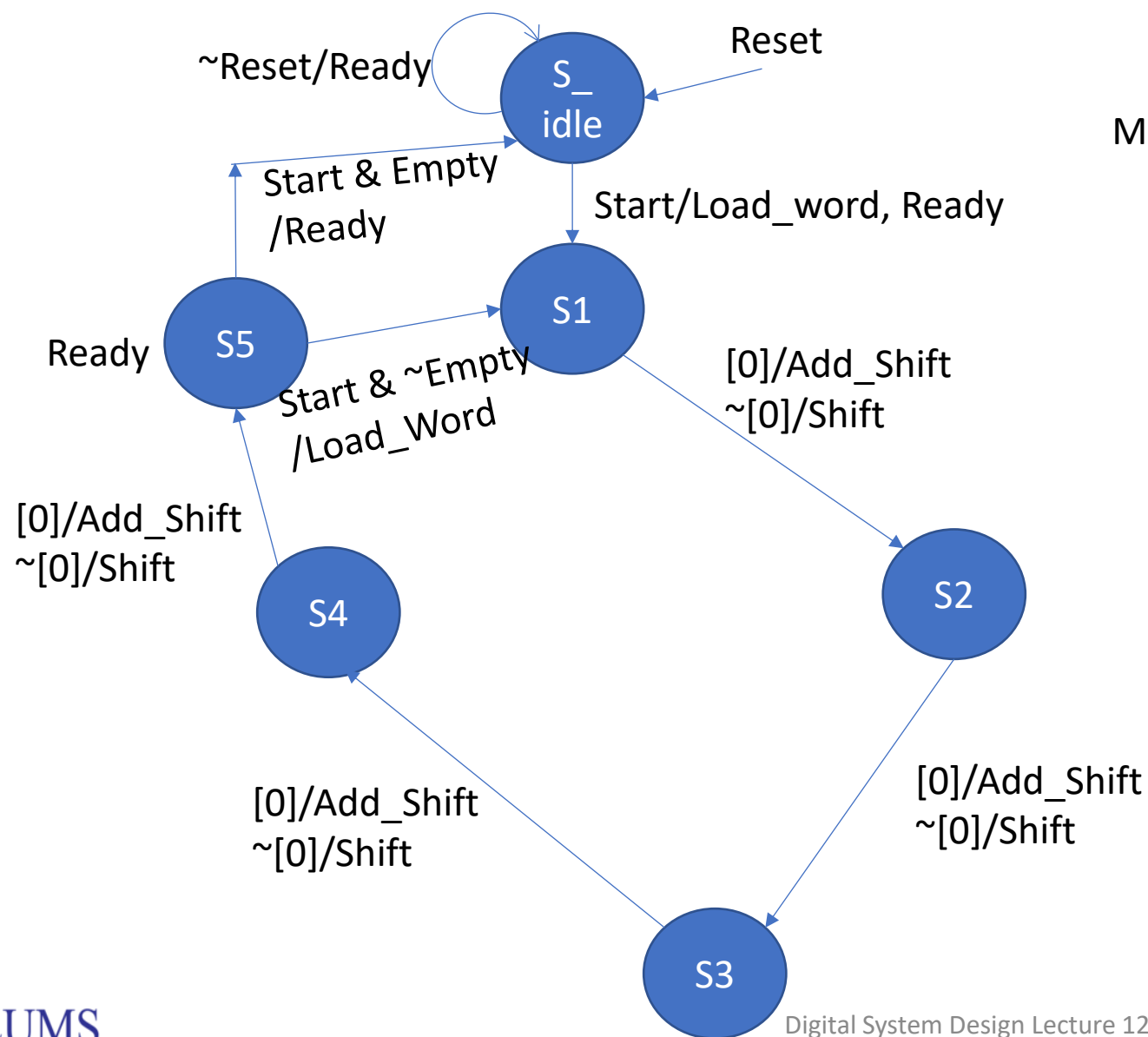


Machine returns to Idle state after Completion of 4 bit multiplication

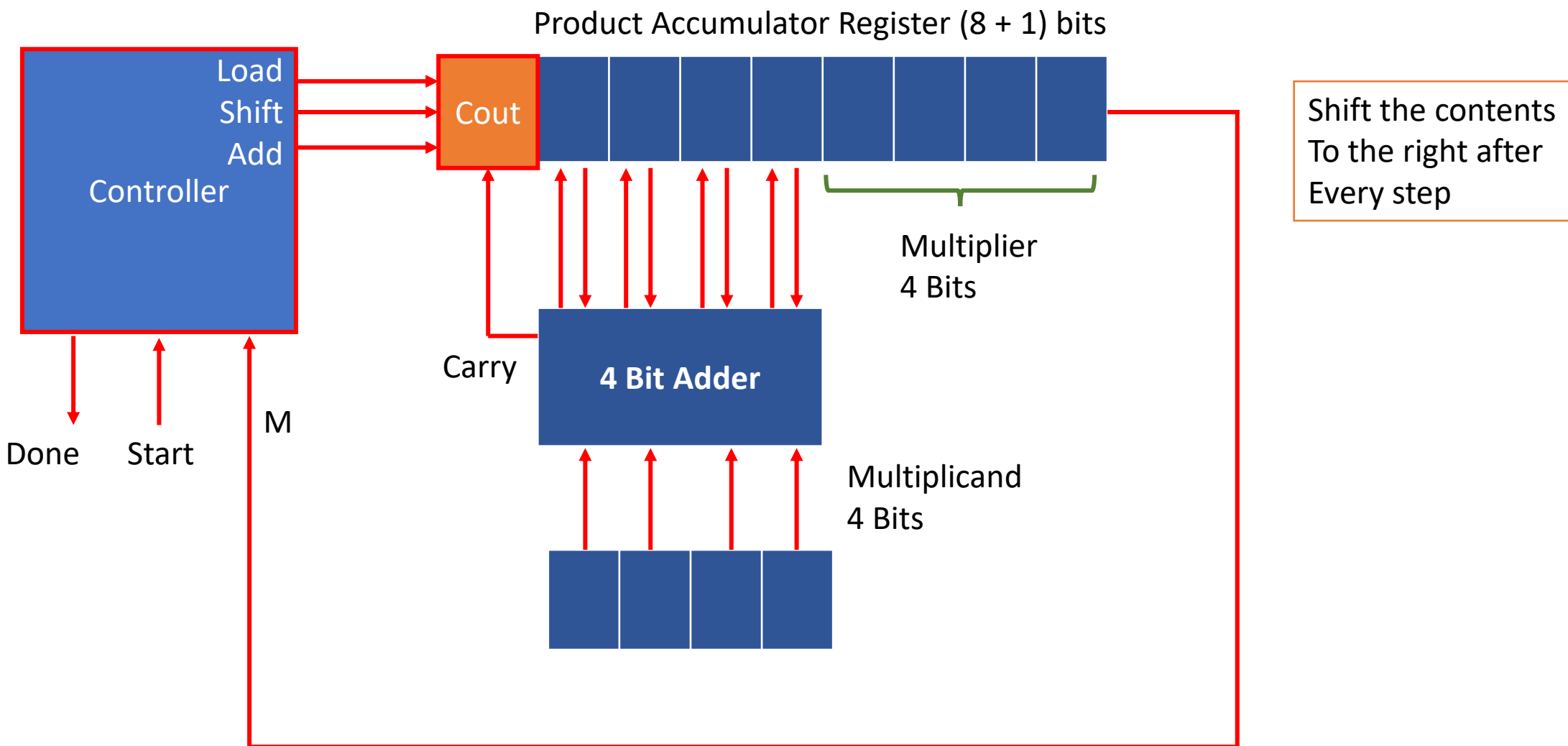
# Improvement - Sequential Multiplier with Reduced Registers



# STG of Reduced Register Sequential Multiplier



# Example of a 4-bit Serial Parallel Multiplier



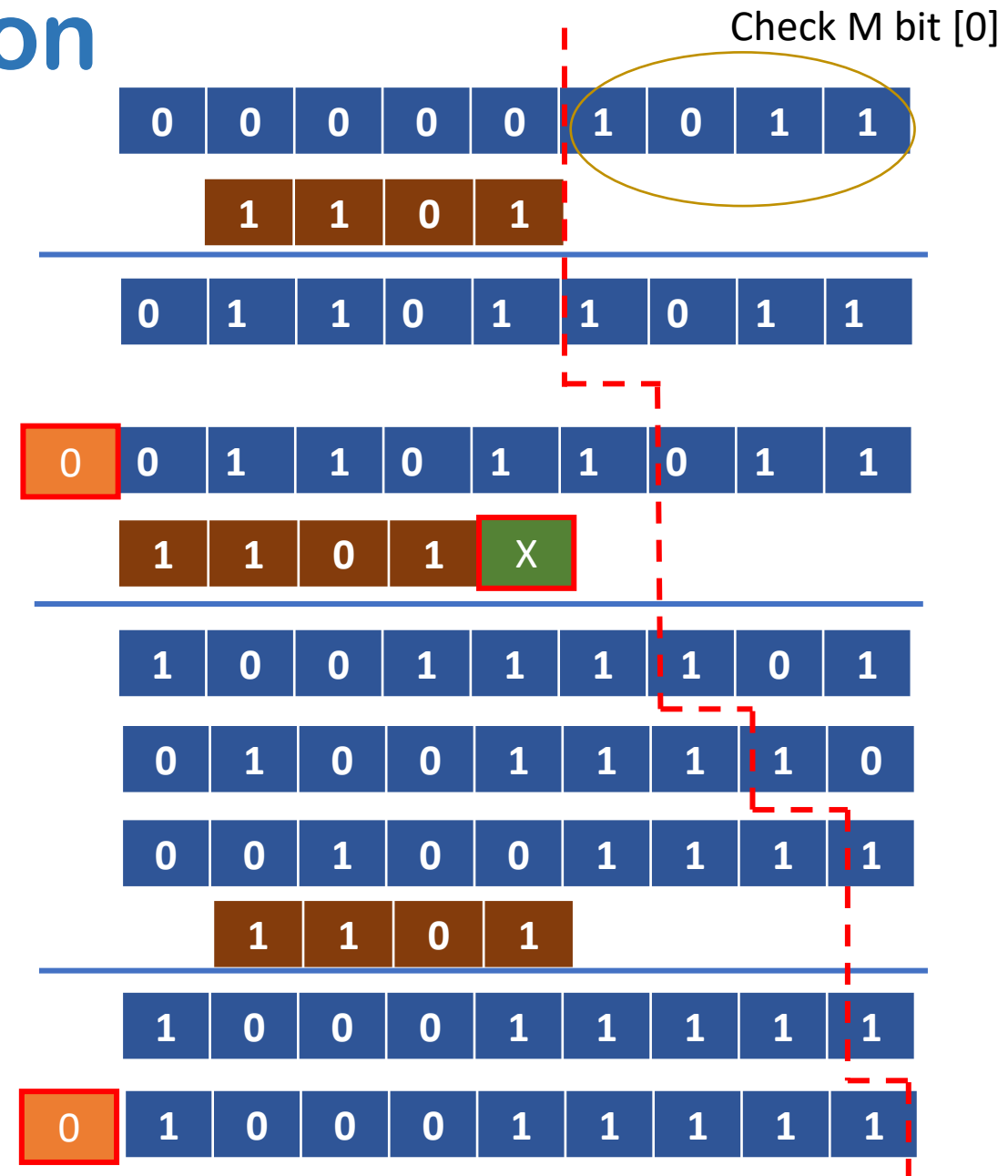
# Example continued

|   |   |   |   |   |   |   |   |                                   |
|---|---|---|---|---|---|---|---|-----------------------------------|
|   |   |   |   | 1 | 1 | 0 | 1 | Multiplicand                      |
|   |   | x |   | 1 | 0 | 1 | 1 | Multiplier                        |
|   |   |   |   | 1 | 1 | 0 | 1 |                                   |
|   |   | 1 |   | 1 | 0 | 1 | X | Shift Left by one                 |
|   | 1 |   | 0 | 0 | 1 | 1 | 1 | Partial product after first step  |
|   | 0 | 0 | 0 | 0 | 0 | X | X | Another shift left                |
|   |   | 1 | 0 | 0 | 1 | 1 | 1 | Partial product after second step |
|   | 1 | 1 | 0 | 1 | X | X | X | Another shift left                |
| 1 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | Partial product after final step  |

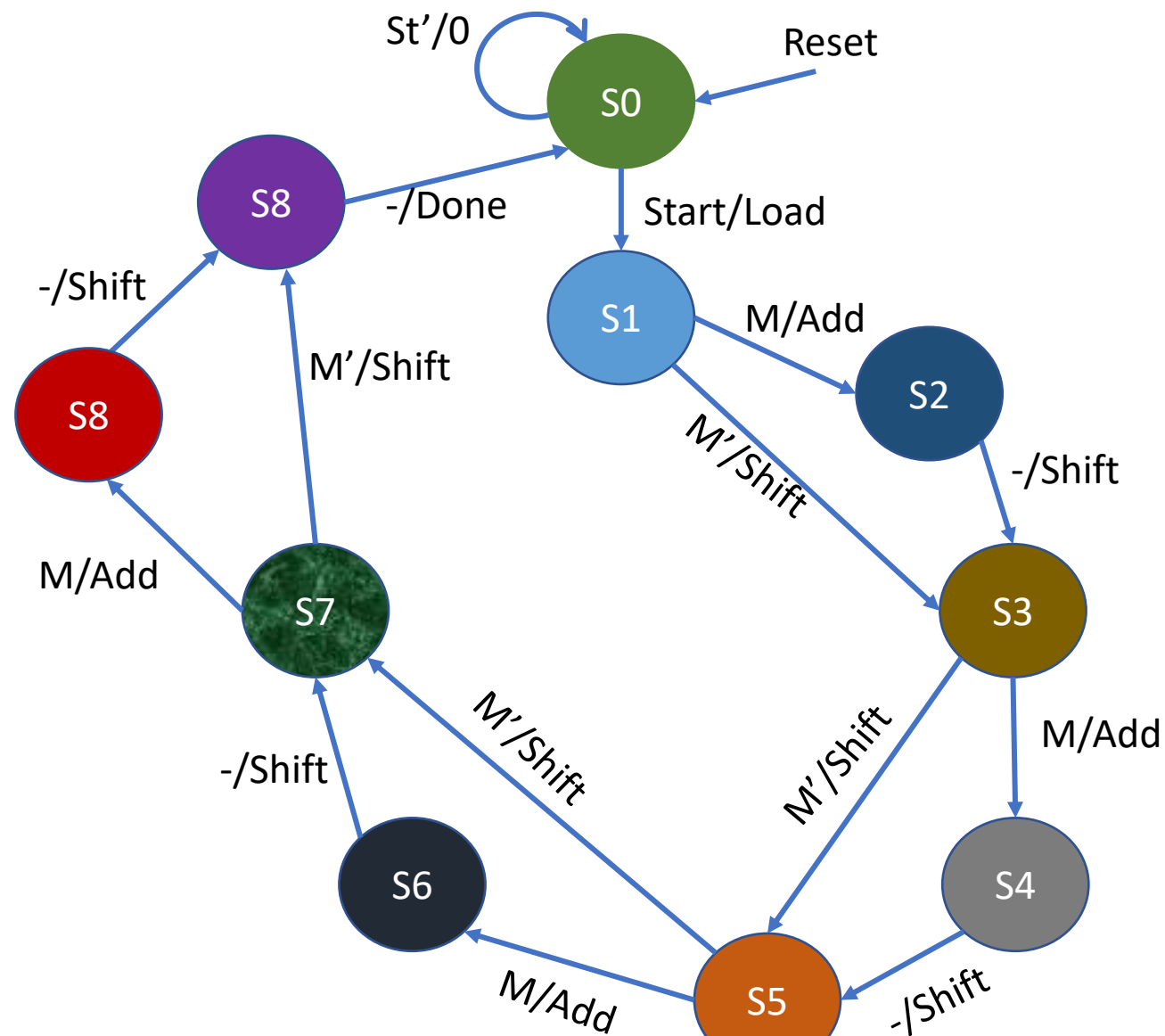
Answer =  $(10001111)_2 = (143)_{10}$

# Multiplier Register Operation

|                                        |
|----------------------------------------|
| Initial Contents of Accumulator        |
| Multiplicand bit [0] is '1'            |
| After Add operation                    |
| After Shift Right                      |
| Next bit M = 1 hence Add               |
| After Add                              |
| After Shift Right                      |
| Next bit M=0, hence Skip Add operation |
| After Shift Right                      |
| Next bit M=1, hence Add                |
| After Addition                         |
| After Shift Right, final answer        |

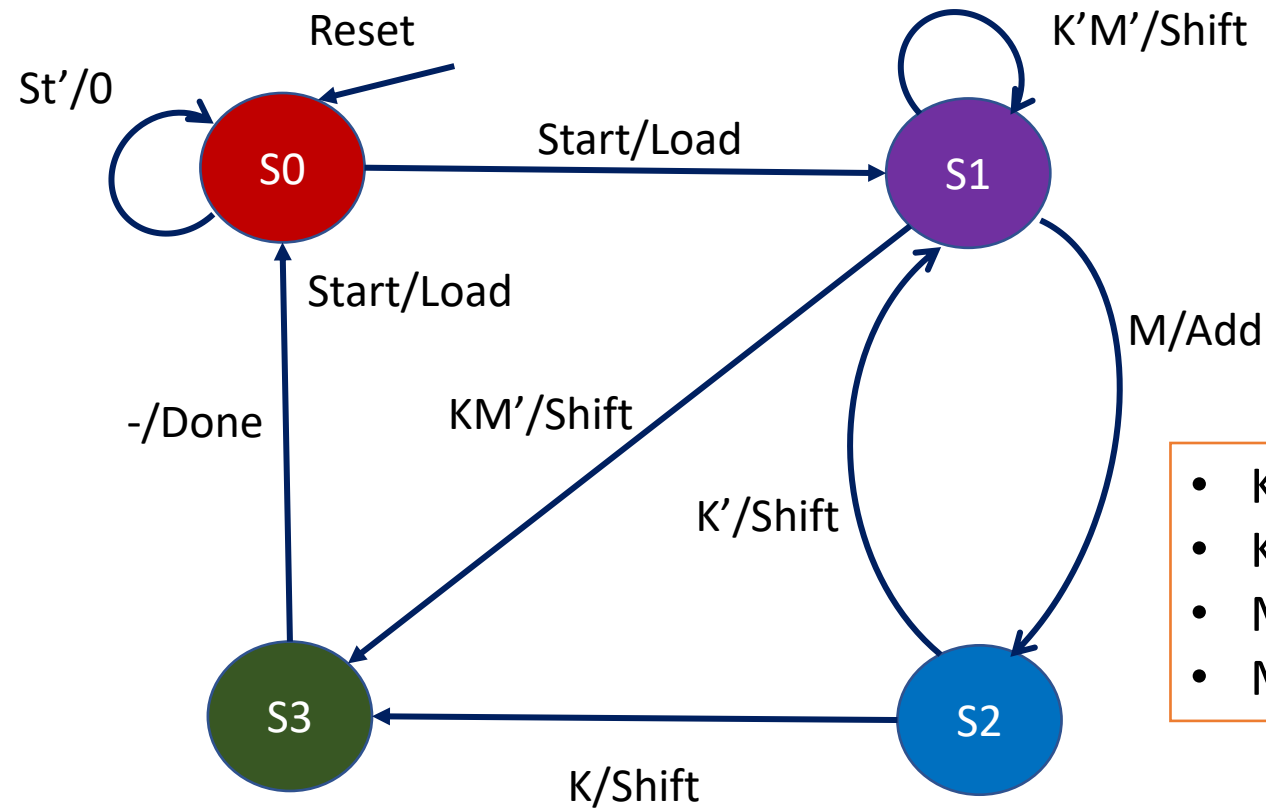


# STG Control Diagram for this Multiplier



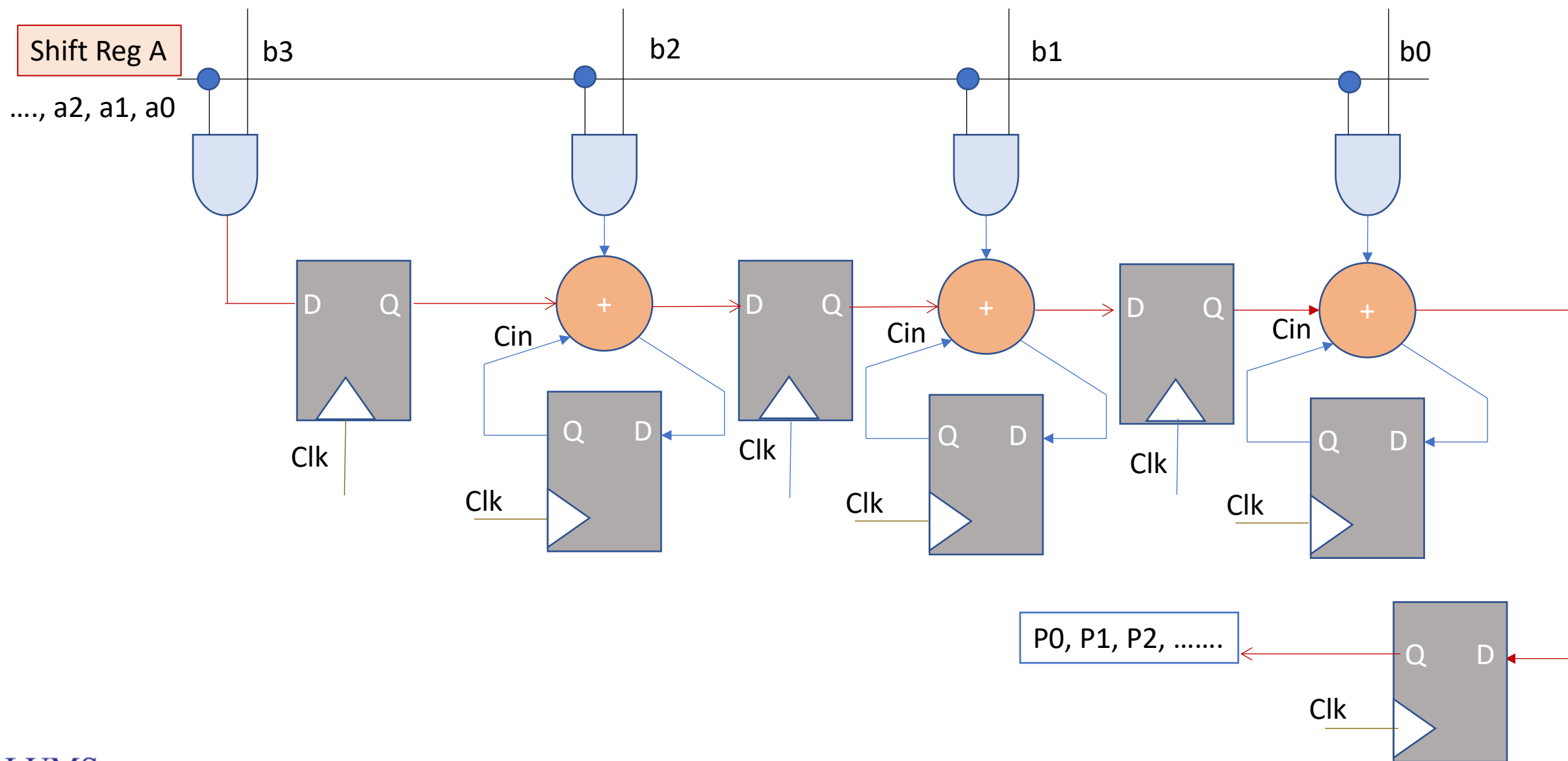
M = relevant bit of multiplier

# Flexible STG for any no. of multiplicand bits



- $K$  = Completion signal generated after  $(n-1)$  shifts
- $K=1$  means one more add (if needed) and shift operation
- $M=0$  means do the final shift
- $M=1$  means add before shift and go to S2

# Parallel-Serial Multiplier - Concept



# Multiplication of Signed Binary Numbers

## Case I: Negative Multiplicand, Positive Multiplier

**Example:**  $-3_{10} \times 6_{10}$

**Bit Assignment:** We assign 8 bits to both numbers.

The product will thus be 16 bits.

$+3 = 0000\ 0011$

Thus 2's Complement =  $-3 = 1111\ 1101$

$+6 = 0000\ 0110$

Sign-bit of the multiplicand must be extended to the word length of the final product before Operating on the 2's Complement words.

This sign-extended multiplicand is used when forming Partial products and accumulated sums.

The result of the multiplication is the 2's Complement of the Product. The final magnitude is found by taking 2's Complement.

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 |
|   |   |   |   |   |   |   | x | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | X |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | X | X |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | 1 | 1 | 0 |

**Remember: Sign Extension to maximum number of bits in datapath**

**Answer = (1111 1111 1111 10 1110)**

**Take 2's Complement, Answer = -(010010) =  $-18_{10}$**

# Multiplication of Fractions

Convert from decimal to binary

$$\left(\frac{3}{4}\right)$$

$$= 0.75$$

$$0.75 \times 2 = 1.5, \text{ keep } 1$$

$$0.5 \times 2 = 1.0, \text{ keep } 1$$

$$0 \times 2 = 0 \text{ keep } 0$$

And only zeros afterwards

$$= 2^{-1} + 2^{-2} + 0 + 0$$

$$= 0.1100; \text{ assigning four fractional bits}$$

# 2's Complement of Binary Fractional Nos.

- Given binary fractional number = (0.1100)
- Method 1:
  - Decide on the number of total bits, eg. 5 bits; Invert all bits; add +1 to LSB
- 2's Complement = 1.0011
- $$\begin{array}{r} \phantom{1.0011} + 1 \\ \hline \phantom{1.0011} = 1.0100 \end{array}$$
- Method 2:
  - Look from right to left; when you Encounter first 1; invert all bits to the left
- For (0.1100), the 2's Complement = (1.0100)

# Question?

- Represent  $9/16$  using five fractional bits:
- Hint:  $9/16 = 0.xxxxx$
- Keep multiplying by 2; if answer is greater than 1, keep 1

- Solve:  $0.562 \times 2 = 1.125$ , keep 1
- $0.125 \times 2 = 0.25$ , keep 0
- $0.25 \times 2 = 0.5$ , keep 0
- $0.5 \times 2 = 1.0$ , keep 1
- $0 \times 2 = 0$ , keep 0, and same for more terms
- Answer = 0.10010 in binary

Finally, 2's Complement of  
 "0.10010" is (look right to left)  
 "1.01110" that is **-9/16**

# Convert Fraction Number to Binary

- Represent 9/16 using five fractional bits:
- Hint:  $9/16 = 0.5625$
- Keep multiplying by 2; if answer is greater than 1, keep 1

- Solve:  $0.5625 \times 2 = 1.125$ , keep 1
- $0.125 \times 2 = 0.25$ , keep 0
- $0.25 \times 2 = 0.5$ , keep 0
- $0.5 \times 2 = 1.0$ , keep 1
- $0 \times 2 = 0$ , keep 0, and same for more terms
- Answer = 0.10010 in binary

Finally, 2's Complement of "0.10010" is (look right to left) "1.01110" that is **-9/16**